

# Binary Trees.

[COMMENT] Complete the Table, Continue the correctness of the minimal length.

Arrays, Linked Lists, Heaps, and Hash Tables are all ways to maintain collections. Each offers a different trade-off in query running times. For some of them, we imposed good worst-case bounds, while for others we can only compute expected costs (Hash Tables), detail are supplied in Table 5.1. The obvious question is whether one could guarantee both fast access and update running times in the worst-case.

In the following two lectures, we will see that using Balanced Binary Search Trees, one can provide both access and insertion/deletion in  $O(\log n)$ .

| Data Structure | Access | Max/min     | Insert/Delete | Max/min     |
|----------------|--------|-------------|---------------|-------------|
| Array          | Data 2 | Data 3      | Data 3        | $O(\log n)$ |
| Linked-List    | $O(n)$ | $O(n)$      | $O(1)$        | $O(\log n)$ |
| Heap           | $O(n)$ | $O(\log n)$ | $O(\log n)$   | $O(\log n)$ |
| Hash Table     | $O(n)$ | $O()$       | $O()$         | $O(\log n)$ |

Table 5.1: Running times respecting to data structure and task

## 1 Binary Trees and How to Encode Them

We have already seen, in heaps, that organizing our data in a graph-like structure can offer a speed advantage. For future applications, and in particular for maintaining data in sorted order, we will have to encode our data using binary trees. In contrast to heaps, these trees may not be almost complete and also have to support pointer manipulations, specifically placing a binary tree as a left or right subtree of a given node. To enable this, we will have to treat the **right**, **left**, and **parent** as variables, in contrast to heaps where they are determined completely by the node index. We begin this section by stating definitions.

### Definition 1.

1. Binary Tree: A tree in which any vertex has at most two children.

2. A descendant of vertex  $x$  is a vertex in the subtree whose root is  $x$ . A left descendant of vertex  $x$  is either a vertex in the subtree whose root is the left child of  $x$ , or  $x$ 's left child.
3. An ancestor of  $x$  is a vertex to which  $x$  belongs as a descendant.
4. A leaf is a vertex without children.
5. Height of vertex  $x$  is the length of the longest simple path (without cycles) between  $x$  and one of the leaves. The height of leaf is 0.
6. Height of the tree is the height of its root, which is usually denoted by  $h$ .

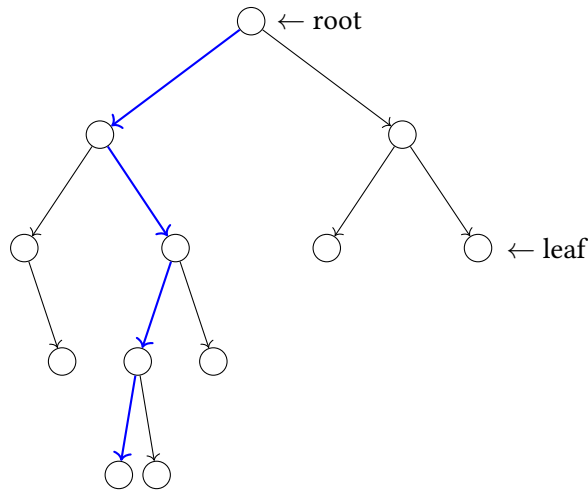


Figure 5.1: Example of a binary tree with 12 vertices. In blue, one of the longest paths from the root to one of the leaves, which is of length 4.

**Question.** What are the maximal and minimal heights of binary trees over  $n$  vertices? **Answer.** The maximal height is  $n - 1$ , and matches to trees in which any vertex has at most one child. The minimal height is  $\log n$ . Let's prove it by induction.

Denote by  $H_n$  the minimal height of a binary tree with  $n$  vertices. Observe that  $H_{n+1} \geq H_n$ . That's because adding a vertex to an  $n$ -size binary tree can't shorten any of the paths from root to leaves, particularly not the longest one.

1. Base.  $n = 1$ , The tree consists of a single vertex, which is both the root and a leaf. Thus, the longest path is of length 0, and also  $\log 1 = 0$ .
2. Assumption. Assume the correctness of the claim for any  $m < n$ .
3. Step. Consider a binary tree over  $n$  vertices with minimal height. The longest path consists of an edge between the root and one of its children. Denote

by  $x$  the size of the root's left subtree, so the size of root's right subtree is  $n - x$ . Thus we get:

$$H_n \geq 1 + \max(H_x, H_{n-x})$$

By the monotonic property of  $H_n$ , we have that the above is minimized when both  $H_x = H_{n-x} = H_{n/2}$  if  $n$  is even, or  $\max(x, n - x) = \lceil n/2 \rceil$  when  $n$  is odd. So we get:

$$\begin{aligned} &\geq 1 + H_{\lceil n/2 \rceil} \\ &= 1 + \log(\lceil \frac{n}{2} \rceil) = \log(n + 1) \end{aligned}$$

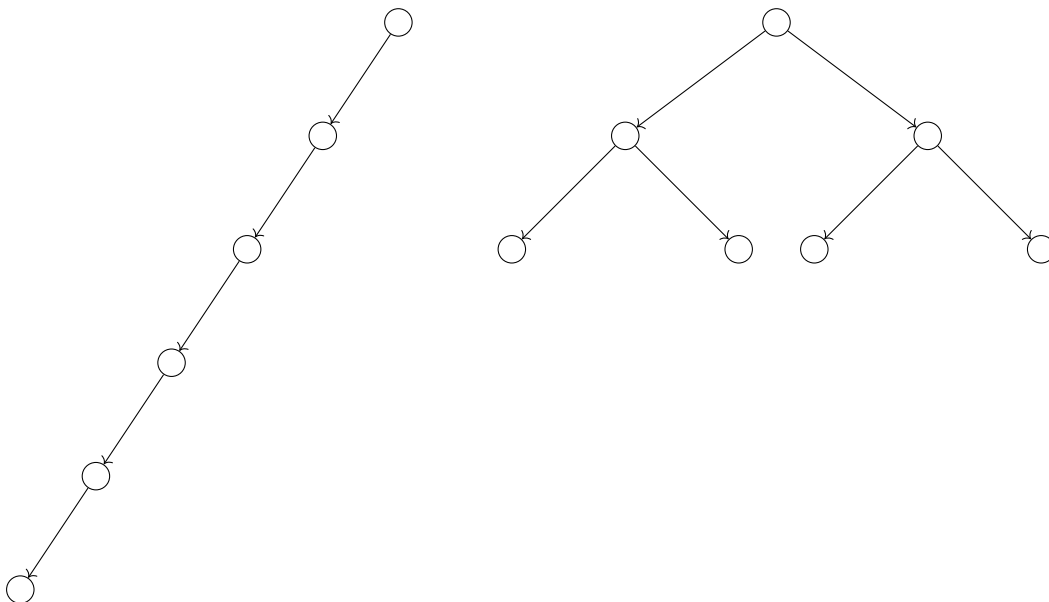


Figure 5.2: On the left, a binary tree with 6 vertices at height 5, while on the right, a binary tree with 7 vertices at height 2.

## 1.1 Encoding

We encode a binary tree by associating a field to each vertex  $x$ , representing its right, left children, and parent. We use the notation  $x.\text{left}$  to refer to the left child of  $x$ , although the physical implementation may differ conceptually. For example, the way binary trees are implemented in Cormen is through 4 arrays. The first stores the value of  $x$ , while the others store pointers of specific types. For instance, the array `LEFT`, where `LEFT. $x$`  stores the left child of  $x$ .

If nothing else has been mentioned, then we can assume that we can add additional fields to the vertices.



**Functionality of BST.**

1.  $\text{Search}(T, \text{key})$ : returns a pointer to the vertex whose key equals key.
2.  $\text{Min}(T)$ : returns a pointer to the vertex with the minimum value in  $T$ .
3.  $\text{Max}(T)$ : returns a pointer to the vertex with the maximum value in  $T$ .
4.  $\text{Predecessor}(x)$ : returns a pointer to the predecessor of  $x$ .
5.  $\text{Successor}(x)$ : returns a pointer to the successor of  $x$ .
6.  $\text{Insert}(T, \text{key})$ : inserts key into  $T$  (creates a new vertex).
7.  $\text{Delete}(T, x)$ : removes  $x$  from  $T$ .
8.  $\text{Inorder}(T)$ : outputs  $T$ 's keys in sorted order.

*Example 1.* Questions from past exams.

1. Write an algorithm that, given a binary search tree  $T$ , returns a max heap containing all the elements of  $T$ . What is the lower bound for this problem? Explain.
2. Write an algorithm that, given a heap  $H$ , returns a binary search tree containing all the elements of  $H$ . What is the lower bound for this problem? Explain.

**Solution.**

1. Remember that any array sorted in reverse order is also a max heap. This is because if  $A_i \geq A_j$  for any  $i < j$ , then it follows that  $A_i \geq A_{2i}, A_{2i+1}$  for any  $i$ . Therefore, we will output the values of  $T$  in reverse order. We do this by applying Inorder in reverse, as given in Algorithm 1. The running time is  $\Theta(n)$  and of course it is also the lower bound (as we must read all the inputs for printing it).

**Data:**  $r$  - a vertex in BST

```

1 if  $r.\text{right}$  is not empty/Null then
2   |   Reverse-Order( $r.\text{right}$ )
3 end
4 Print  $r.\text{key}$ 
5 if  $r.\text{left}$  is not empty/Null then
6   |   Reverse-Order( $r.\text{left}$ )
7 end
```

**Algorithm 1:** Reverse-Order

2. We will initialize an empty binary search tree and then add the elements to it one by one. Each insertion will cost the current height of the tree, so it might sum up to  $\Theta(n^2)$ . However, next week we will see how those insertions could be done in a way that preserves the balance of the tree, namely guaranteeing that the height of the tree will remain logarithmic.

*Example 2.* Adding fields to BSTs. We would like to support  $k$ -smallest element extraction. Suggest a field that will be used for computing the  $k$ -smallest element, explain how to maintain it, and write an algorithm that extracts the  $k$ -smallest element.

**Solution.** We will associate with each vertex of the tree a field that counts the number of nodes whose keys are lower than it (the size of its left subtree). Let's denote it as left-size. The code is given in Algorithm 2. Guideline for proving correctness: Consider the root, and let's separate into cases based on how many elements the root is greater than.

1. If the root is strictly greater than  $k - 1$  elements, then the  $k$ -smallest element is in its left subtree and is also the  $k - 1$  smallest element in that subtree.
2. If the root is greater than strictly fewer than  $k - 1$  elements, then it is clear that the  $k$ -smallest element is also greater than it and located in its right subtree. However, in contrast to the previous case, here the order of the  $k$ -smallest element of the whole tree in the subtree will be the substitution between  $k$  and the number of nodes in the  $\{\text{left-subtree} \cup \text{root}\}$ .
3. The third case is when the root is greater than exactly  $k - 1$  elements, which by definition sets it as the  $k$ -smallest element.

In the first two cases, we are calling Get-Order on trees (BST with our additional field) with smaller height. Therefore, it is clear how one would prove correctness by induction. We will see in the next recitation how to maintain the tree, update the field, and guarantee that the height of the tree will remain logarithmic.

**Data:**  $r$  - a vertex in BST,  $k$ -stat parameter

```

1  $m \leftarrow r.\text{left-size}$ 
2 if  $m = k - 1$  then
3   | return  $r.\text{key}$ 
4 end
5 if  $m > k - 1$  then
6   | return Get-Order( $r.\text{left}$ ,  $k$ )
7 end
8 if  $m < k - 1$  then
9   | return Get-Order( $r.\text{right}$ ,  $k - (m + 1)$ )
10 end
```

**Algorithm 2:** Get-Order

### 3 Appendix - BST Methods Source: A Mixture of Cormen, Wikipedia, and Codeforces

**Data:**  $T$  - tree, key - key to search for

**Result:** pointer to vertex with key equal to key

```

1 if  $T$  is empty then
2   | return null;
3 end
4 if  $T.key$  equals key then
5   | return  $T$ ;
6 end
7 if key is less than  $T.key$  then
8   | return Search( $T.left$ , key);
9 end
10 else
11   | return Search( $T.right$ , key);
12 end

```

**Algorithm 3:** Search

**Data:**  $T$  - tree

**Result:** pointer to vertex with minimum value in  $T$

```

1 if  $T$  is empty then
2   | return null;
3 end
4 if  $T.left$  is empty then
5   | return  $T$ ;
6 end
7 return Min( $T.left$ );

```

**Algorithm 4:** Min

**Data:**  $T$  - tree

**Result:** pointer to vertex with maximum value in  $T$

```

1 if  $T$  is empty then
2   | return null;
3 end
4 if  $T.right$  is empty then
5   | return  $T$ ;
6 end
7 return Max( $T.right$ );

```

**Algorithm 5:** Max

**Data:**  $x$  - vertex

**Result:** pointer to predecessor of  $x$

```

1 if  $x.left$  is not empty then
2   | return Max( $x.left$ );
3 end
4  $y \leftarrow x.parent$ ;
5 while  $y$  is not empty and  $x$  is  $y.left$  do
6   |  $x \leftarrow y$ ;
7   |  $y \leftarrow y.parent$ ;
8 end
9 return  $y$ ;

```

**Algorithm 6:** Predecessor

**Data:**  $x$  - vertex

**Result:** pointer to successor of  $x$

```

1 if  $x.right$  is not empty then
2   | return Min( $x.right$ );
3 end
4  $y \leftarrow x.parent$ ;
5 while  $y$  is not empty and  $x$  is  $y.right$  do
6   |  $x \leftarrow y$ ;
7   |  $y \leftarrow y.parent$ ;
8 end
9 return  $y$ ;

```

**Algorithm 7:** Successor

**Data:**  $T$  - tree, key - key to insert

**Result:** inserts key into  $T$

```

1 newNode  $\leftarrow$  create new vertex with key  $key$ ;
2  $y \leftarrow$  null;
3  $x \leftarrow T$ ;
4 while  $x$  is not empty do
5   |  $y \leftarrow x$ ;
6   | if  $key$  is less than  $x.key$  then
7   |   |  $x \leftarrow x.left$ ;
8   | end
9   | else
10  |   |  $x \leftarrow x.right$ ;
11  | end
12 end
13 newNode.parent  $\leftarrow y$ ;
14 if  $y$  is null then
15   |  $T \leftarrow newNode$ ;
16 end
17 else if  $key$  is less than  $y.key$  then
18   |  $y.left \leftarrow newNode$ ;
19 end
20 else
21   |  $y.right \leftarrow newNode$ ;
22 end
```

**Algorithm 8:** Insert

**Data:**  $T$ : The input tree

```

1 if  $T$  is not empty then
2   | Inorder( $T.left$ );
3   | Output  $T.key$ ;
4   | Inorder( $T.right$ );
5 end
```

**Algorithm 9:** Inorder( $T$ )